Solving Elliptic PDE's using Physics-Informed Neural Networks

Aditya Saroha

2025 - Present

Abstract

This report contains the solutions to some of the problems which I have solved using PINN's, most of the solutions by PINN's have been converged but for the lid driven caivty case(steady-state, incompressible fluid), dimensionless 2D `Navier Stokes Equation's` solution is not converging well. Each problem's setup is explained and the relevant `Residual equations` are also derived. `GenAI` disclosure is given at the end of this report.

1 Problem 1: Projectile Motion as a Second-Order ODE

1.1 Governing Equation

Consider the vertical motion of a ball under constant gravitational acceleration g. if we neglect air resistance, then the motion is governed by Newton's second law of motion:

$$\frac{d^2y}{dt^2} = -g$$

This is a second-order ordinary differential equation defined for $t \in (0, T)$.

1.2 Initial Conditions

The initial conditions for this arrangement is:

$$y(0) = 0, \tag{1}$$

$$\frac{dy}{dt}(0) = u_0 \sin(\theta) = 0, \tag{2}$$

where u_0 is the initial velocity magnitude and θ is the launch angle.

1.3 Analytical Solution

The closed-form solution is given by:

$$y(t) = u_0 \sin(\theta)t - \frac{1}{2}gt^2$$

1.4 PINN Residual Formulation

Let $y_\theta(t)$ denote the neural network approximation. The physics residual is defined as:

$$\mathcal{R}_\theta(t) = \frac{d^2 y_\theta}{dt^2} + g$$

The residual loss is:

$$\mathcal{L}_{res} = \frac{1}{N_r} \sum_{i=1}^{N_r} |\mathcal{R}_\theta(t_i)|^2$$

The initial condition loss is:

$$\mathcal{L}_{ic} = |y_\theta(0)|^2 + \left| \frac{dy_\theta}{dt}(0) - u_0 \sin(\theta) \right|^2$$

The total loss becomes:

$$\mathcal{L} = \lambda_r \mathcal{L}_{res} + \lambda_{ic} \mathcal{L}_{ic}.$$

1.5 Results & Discussion

For this problem a very small feedforward neural network architecture was designed with just 1 hidden layer with 64 neurons. Such a small network was able to approximate the solution since there was very less complexity involved.

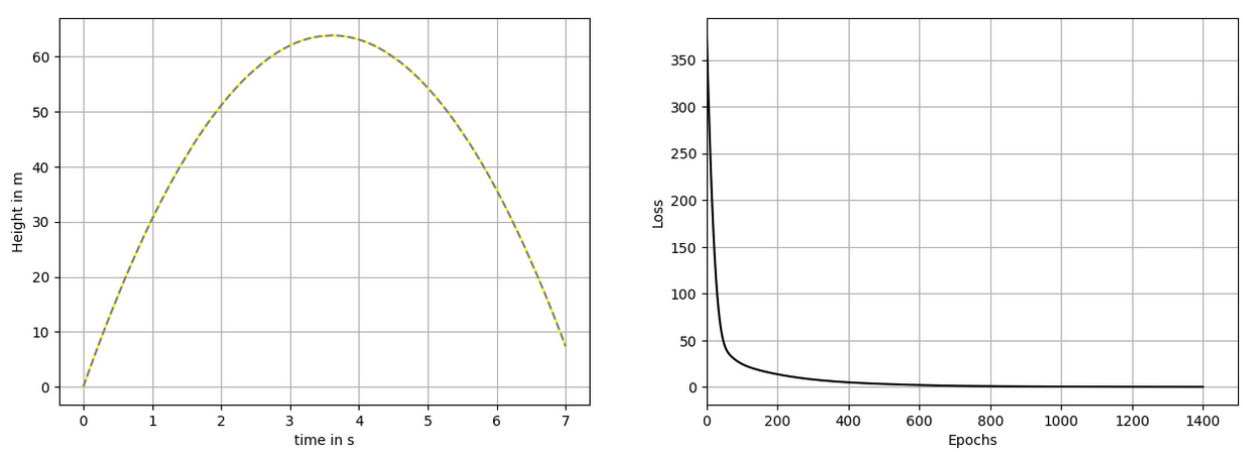

Figure 1: Image one is the comparison of the analytical solution vs PINN approximation and second image is the loss curve over 1499 epochs

The solution looks acceptable to me with no scope of improvements since it has converged (two quantities are equivalent when they are not changing its values upto 3-4 decimal places.)

2 Problem 2: Poisson equation on a 2D domain with boundary conditions

2.1 Governing Equation

Let us consider a steady-state heat conduction on a square plate in only 2 dimensions, then the heat conduction is governed by poisson equation as:

$$\nabla^2 T(x, y) = f(x, y)$$

This is a second order partial differential equation where, $(x, y) \in \Omega$ and $\Omega \in (0, 1) \times (0, 1)$. Also T is function which we want to find and f is the source term, let the soruce term be:

$$f(x, y) = -2\pi^2 \cdot \sin(\pi x) \sin(\pi y)$$

We know that:

$$\nabla^2 T(x, y) = \frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2}$$

2.2 Boundary conditions

We will have the `Drichlet` boundary conditions in this setup, so:

$$T(x, y) = T(0, y) = T(x, 0) = 0$$
$$y = 0, \quad x \in (0, 1)$$
$$y = 1, \quad x \in (0, 1)$$
$$x = 0, \quad y \in (0, 1)$$
$$x = 1, \quad y \in (0, 1)$$

2.3 Analytical solution

The solution is given by:

$$T(x, y) = \sin(\pi x) \sin(\pi y)$$

2.4 PINN Residual Formulation

Let $T_\theta(x, y)$ denote the neural network approximation, then the physics residual is defined as:

$$\mathcal{R}_\theta(x, y) = \nabla^2 T_\theta(x, y) \ - f(x, y)$$

$$\mathcal{R}_\theta(x, y) = \frac{\partial^2 T_\theta(x, y)}{\partial x^2} \ + \frac{\partial^2 T_\theta(x, y)}{\partial y^2} \ + (-2\pi^2 \sin(\pi x) \sin(\pi y))$$

So the Residual Loss is:

$$\mathcal{L}_{pde} = \frac{1}{n} \cdot \sum_{i=1}^{n} |\mathcal{R}_\theta(x, y)|^2$$

The boundary loss is:

$$\mathcal{L}_b = \frac{\mathcal{L}_{right} + \mathcal{L}_{left} + \mathcal{L}_{top} + \mathcal{L}_{bottom}}{4}$$

$$\mathcal{L}_{right} = \frac{1}{n} \cdot \sum_{i=1}^{n} |T_\theta(0, y)|^2$$

$$\mathcal{L}_{left} = \frac{1}{n} \cdot \sum_{i=1}^{n} |T_\theta(1, y)|^2$$

$$\mathcal{L}_{top} = \frac{1}{n} \cdot \sum_{i=1}^{n} |T_\theta(x, 1)|^2$$

$$\mathcal{L}_{bottom} = \frac{1}{n} \cdot \sum_{i=1}^{n} |T_\theta(x, 0)|^2$$

Notice the division by 4 in the equation $\mathcal{L}_b$, it is due to the mean in all the 4 boundary conditions, if we ignore it, then λ_b will have its value multiplied by 4 unnecessarily.
The total loss becomes:

$$\mathcal{L} = \lambda_{dpe}\mathcal{L}_{pde} + \lambda_b\mathcal{L}_b$$

2.5 Results & Discussion

For this problem a very small feed-forward neural network was designed with 3 hidden layers and an additional normalization layer which re-samples the inputs between $[-1, 1]$. For this case specifically, a combination `sinh` and `tanh` activation functions are used within the hidden layers and in the outputs no activation function is used.

```
from tensorflow.keras import layers
>>> layers.Lambda(lambda x: x * 2 - 1)
```

Also the custom `sinh` function:

```
import tensorflow as tf
def sinh_activation_func(x: tf.tensor) -> tf.tensor:
    return tf.math.sinh(x)
```

The network was trained for 2500 `epochs` with a learning of $1e^{-}4$, initially the loss curve did show overshooting, but after reducing the `learning rate` and tweaking the `clipnorm` the training became smooth and finally converged.

We can see in the loss curve that the `bc_loss` spikes for almost ~ 300 epochs and then slowly decreases. This behavior is due to loss weighting, since $\lambda_{bc} \ll \lambda_{pde}$.

As far as sampling strategy is concerned, I have used uniform sampling strategy for this problem, as the solutions converged there was no need to use more complex sampling strategy for training or inference.

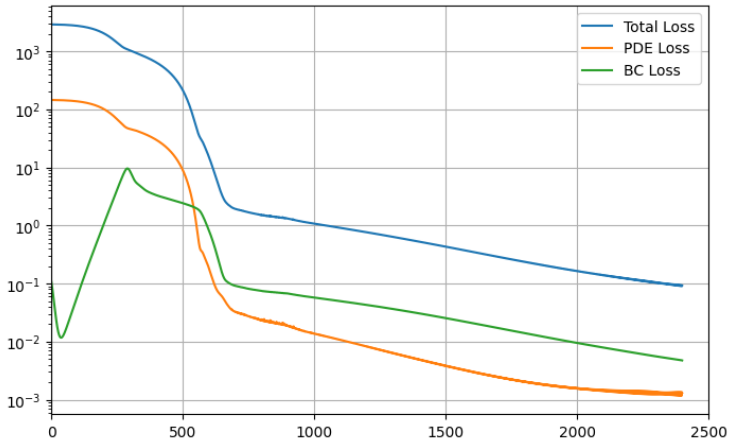

Figure 2: Loss curve over 2500 epochs

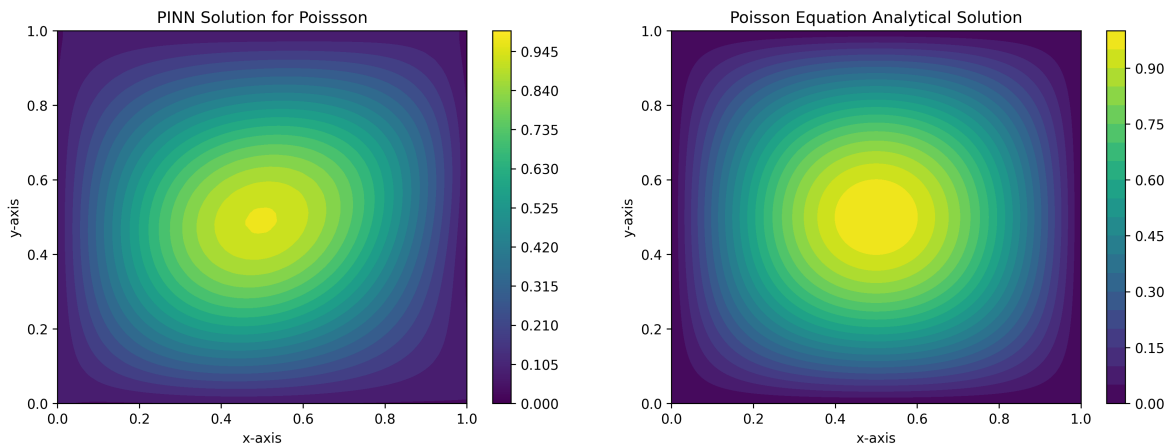

Figure 3: The first image shows the solution approximated by the PINN and the second image shows the analytical solution

The above solution of the PINN looks converged by using only 2 hidden layered network, apart from that better solutions can be obtained over the domain Ω by using networks with higher representation power.

3 Problem 3: 1D ODE with user-chosen forcing term

3.1 Governing Equation

Let us consider a second-order ordinary differential equation:

$$\frac{d^2y}{dx^2} = f(x)$$

defined for $x \in (0, 1)$ and with `drichlet` boundary conditions in only 1 dimension.
Let the forcing term be:

$$f(x) = x^3$$

3.2 Boundary Conditions

The boundary conditions for this problem is given by:

$$y(0) = 0$$

$$y(1) = 0$$

3.3 Analytical Solution

Integrating w.r.t x on both sides:

$$\int \frac{d^2y}{dx^2} dx = \int x^3 dx$$

$$\frac{dy}{dx} = \frac{x^4}{4} + C_1$$

Integrating again w.r.t x on both sides:

$$\int \frac{dy}{dy} dx = \int (\frac{x^4}{4} + C_1) dx$$

$$y = \frac{1}{4} \int x^4 dx + \int C_1 dx$$

$$y = \frac{x^5}{20} + C_1 x + C_2$$

After putting $y(0) = 0$ and $y(1) = 0$ we get $C_2 = 0$ and $C_1 = \frac{-1}{20}$
Therefore the exact solution is:

$$y(x) = \frac{x^5}{20} - \frac{x}{20}$$

3.4 PINN Residual Formulation

Let $y_\theta(x)$ denote the neural network approximation, then the physics residual is defined as:

$$\mathcal{R}_\theta(x) = \frac{d^2 y_\theta}{dx^2} - x^3$$

So the Residual Loss is:

$$\mathcal{L}_{ode} = \frac{1}{n} \sum_{i=1}^{n} |\mathcal{R}_\theta(x)|^2$$

The boundary Loss is:

$$\mathcal{L}_b = \frac{1}{n} \sum_{i=1}^{n} |y_\theta(0) + y_\theta(1)|^2$$

Therefore the toal loss is:

$$\mathcal{L} = \lambda_{ode} \mathcal{L}_{ode} + \lambda_b \mathcal{L}_b$$

3.5 Results & Discussion

For this problem too, a very small feed-forward neural network was designed with only 2 hidden layers with 32 neurons each and an additional normalization layer which re-samples the input between $[0, 1]$. I have used `tanh` activation functions in both the layers, and in the output no activation function is used.

This network was trained for over 512 `epochs` with a learning rate of $1e^{-}4$ and no overshooting in the loss curve was seen. The sampling strategy was uniform, and the loss-weighting was manually set by weighing the boundary loss more than the `pde` loss.

The solution approximated by PINN looks acceptable by keeping in mind the size off network used to produce this solution, a more accurate solution can be obtained over the domain (x, y) using a slightly bigger network and using better loss-weighing strategy.

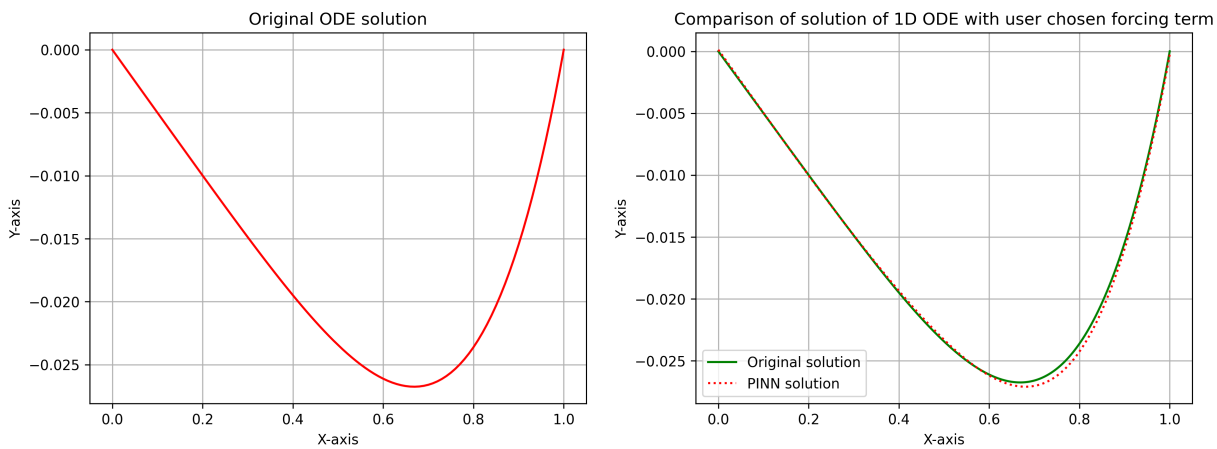

Figure 4: The first image shows the analytical solution and the second image shows the comparison of analytical solution and solution approximated by PINN

4 Problem 4: 2D Steady-State Incompressible Flow in a Lid-Driven Cavity

4.1 Governing Equation

Below you can see the geometry of the lid driven cavity, apart from the top wall all other walls are fixed and cannot move. The top wall is a lid with a velocity in the x direction $1m/s$, therefore $U_X = 1$ (we will be solving this problem in dimension less form).

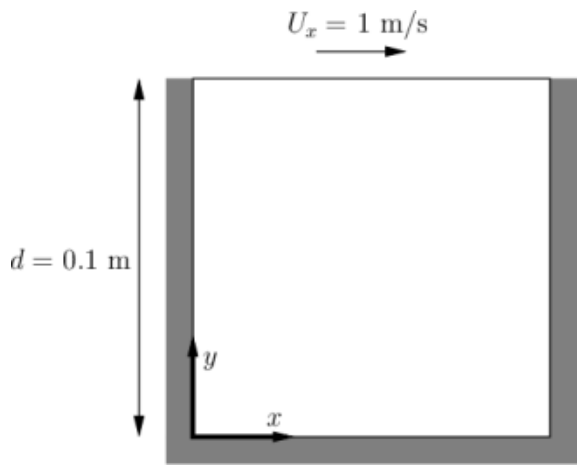

Figure 5: Geometry of the lid driven cavity (source)

As we know that, any type of fluid is governed by the Navier-Stokes equation hence this case will be governed by the incompressible steady-state dimensionless 2D navier-stokes equation:

$$\nabla \cdot \mathbf{u}^* = 0$$

$$(\mathbf{u}^* \cdot \nabla)\mathbf{u}^* = -\nabla p^* \ + \frac{1}{R_e}\nabla^2 \mathbf{u}^*$$

In 2D dimensionless form $\mathbf{u}^* = (u^*, v^*)$ and other quantities are represented as:

$$u^* = \frac{u}{U}, \quad v^* = \frac{v}{U}, \quad p^* = \frac{p}{\rho U^2}, \quad x^* = \frac{x}{L}, \quad y^* = \frac{y}{L}$$

here, L is the cavity length, U is the lid velocity and R_e is the Reynolds number.(in this case we will be using $R_e = 100.0$)

4.2 Analytical Solution

An exact solution to the lid driven cavity problem does not exist for even R_e as low as 100 due to the highly complex nature of the navier-stokes equation. The PINN solution can be compared by the ghia. et. el.(1982), which is the benchmark for High-R_e solutions. But for this case we will be using a open source software for CFD simulation i.e. `openFoam` for solving this case.

The cavity is modeled as a square domain of side $L = 0.1$, with the top lid moving at $u = 1$ and all other walls satisfying no-slip conditions ($u = v = 0$). The solution obtained from OpenFOAM is solved using the `Finite Volume Method` and is used to validate the physics-informed neural network (PINN) predictions, by comparing the velocity components u^*, v^* and the pressure p^* at selected points inside the cavity and also by comparing the contour plots of the fields.

This reference solution provides another way to assess the accuracy and convergence of the PINN approximation.

4.3 Residual Formulation

Let $\mathbf{u}^*_\theta(x^*, y^*)$ denote the neural network approximation, then then the general physics residual is defined as:

$$\mathcal{R}_\theta(x^*, y^*) = (\mathbf{u}^* \cdot \nabla)\mathbf{u}^* + \nabla p^* - \frac{1}{R_e}\nabla^2 \mathbf{u}^*$$

The x^*-component Residual Equation is given as:

$$\mathcal{R}_{\theta_1}(x^*, y^*) = u^* \frac{\partial u^*}{\partial x^*} + v^* \frac{\partial u^*}{\partial y^*} + \frac{\partial p^*}{\partial x^*} - \frac{1}{R_e}\left(\frac{\partial^2 u^*}{\partial x^{*2}} + \frac{\partial^2 u^*}{\partial y^{*2}}\right)$$

The y^*-component Residual Equation is given as:

$$\mathcal{R}_{\theta_2}(x^*, y^*) = u^* \frac{\partial v^*}{\partial x^*} + v^* \frac{\partial v^*}{\partial y^*} + \frac{\partial p^*}{\partial y^*} - \frac{1}{R_e}\left(\frac{\partial^2 v^*}{\partial x^{*2}} + \frac{\partial^2 v^*}{\partial y^{*2}}\right)$$

Also the Mass conversation or Continuity Residual Equation is given as:

$$\mathcal{R}_{\theta_3}(x^*, y^*) = \frac{\partial u^*}{\partial x^*} + \frac{\partial v^*}{\partial y^*}$$

Therefore the combined the Residual Equation is defined as:

$$\mathcal{G}_\theta(x^x, y^*) = \mathcal{R}_{\theta_1}(x^*, y^*)^2 + \mathcal{R}_{\theta_2}(x^*, y^*)^2 + \mathcal{R}_{\theta_3}(x^*, y^*)^2$$

So the Residual loss is given by:

$$\mathcal{L}_{pde} = \frac{1}{n}\sum_{i=1}^{n} |\mathcal{G}_\theta(x^x, y^*)|$$

The boundary loss is calcualted in this manner, we will calculate separately the boundary loss for the top, bottom, left and right walls using our boundary conditions of the vertical and horizontal velocities:

$$\mathcal{L}_{top} = \frac{1}{n}\sum_{i=1}^{n} |(U_\theta^*(x^*, y^*) - U_0^*(x^*, 1))^2 + (V_\theta^*(x^*, y^*) - V_0^*(x^*, 1))^2|$$

$$\mathcal{L}_{bottom} = \frac{1}{n}\sum_{i=1}^{n} |(U_\theta^*(x^*, y^*) - U_0^*(x^*, 0))^2 + (V_\theta^*(x^*, y^*) - V_0^*(x^*, 0))^2|$$

$$\mathcal{L}_{left} = \frac{1}{n}\sum_{i=1}^{n} |(U_\theta^*(x^*, y^*) - U_0^*(0, y^*))^2 + (V_\theta^*(x^*, y^*) - V_0^*(0, y^*))^2|$$

$$\mathcal{L}_{right} = \frac{1}{n}\sum_{i=1}^{n} |(U_\theta^*(x^*, y^*) - U_0^*(1, y^*))^2 + (V_\theta^*(x^*, y^*) - V_0^*(1, y^*))^2|$$

Where $U_\theta^*(x^*, y^*)$ is the predicted U_x, $V_\theta^*(x^*, y^*)$ is the predicted U_y fetched from the tensor $\mathcal{R}_\theta(x^*, y^*)$ and $U_0^*(1, y^*)$, $U_0^*(x^*, 0)$, $U_0^*(x^*, 1)$, $U_0^*(0, y^*)$, $V_0^*(x^*, 1)$, $V_0^*(x^*, 0)$, $V_0^*(0, y^*)$, $V_0^*(1, y^*)$ are the initial velocities at the boundary.

Therefore the total boundary loss is:

$$\mathcal{L}_b = \frac{\mathcal{L}_{top} + \mathcal{L}_{bottom} + \mathcal{L}_{left} + \mathcal{L}_{right}}{4}$$

Also we can note that there is no means we can calculate the pressure in this setup, since it is present only in ∇p form. There are many algorithms like `SIMPLE` or even a better version `SIMPLEC` which manipulates the equation to derive a pressure equation (also called as pressure corrector, since they first assume the pressure values then solve the continutiy equation in order to satisfy the presure values). OpenFoam's solver like `simpleFoam` or `icoFoam` uses various algorithm like `SIMPLE` or `PISO` to solve the NVS equation. What I mean is this, let us consider the first pass in the network and it predicts values of U_x, U_y, P. When we calculate the residuals we see that, the P will not be present in the equation, instead its gradient will be present in both the residual equation and the backpropagation equation. This means that, W_p will only tell the network how to shape the slope of the pressure field correctly not its absolute position.
So we will additionally add a pressure loss in our setup so that when the network performs back-propogation, it can calcualtes the absolute values of pressure:

$$\mathcal{L}_p = \frac{1}{n}\sum_{i=1}^{n} |(P_\theta^*(x^*, y^*) - 0)^2| \tag{i}$$

Therefore the total loss is:

$$\mathcal{L} = \lambda_{pde}\mathcal{L}_{pde} + \lambda_b\mathcal{L}_b + \lambda_p\mathcal{L}_p$$

4.4 Results & Discussion

For this problem, I have selected a very small neural network with 3 hidden layers and using the *tanh* activation function. The solution is not converged for the given domain, the U_x^* and U_y^* are showing converging behaviour but the approximation by the network is not accurate. I intend to use ALPINNs for adaptive weighting and also use the FFT and better collocation point sampling so that the network can learn better. Also I faced the problem of exploding gradient even after rescaling the L_2 norm of the gradient.

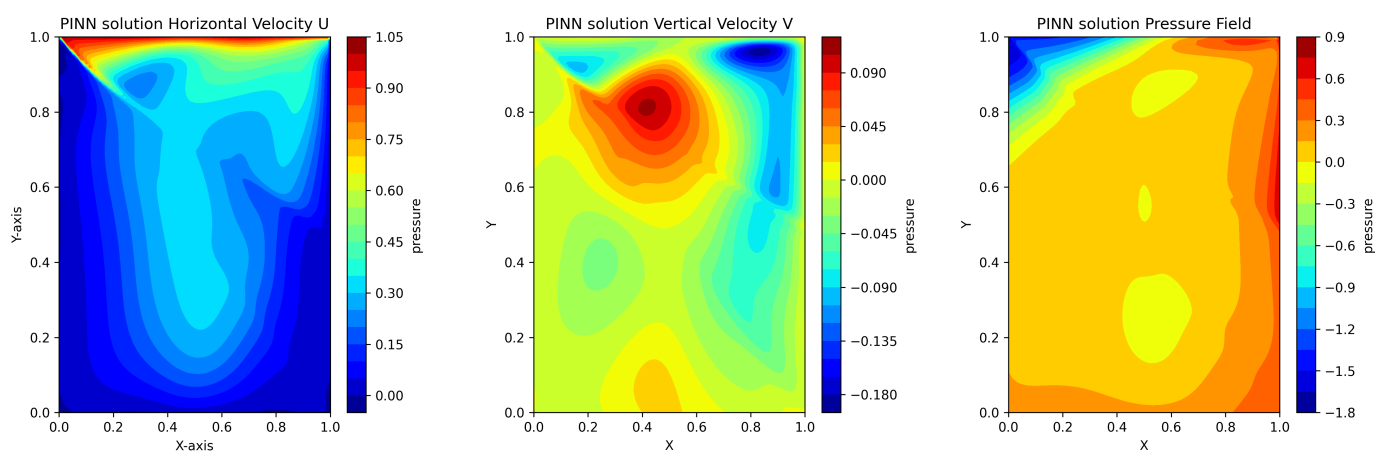

Figure 6: These contour plots shows the solutions approximated by the PINNs with (i) Horizontal velocity - U_x^*, (ii) Vertical Velocity - U_y^* and (iii) Pressure - p^*

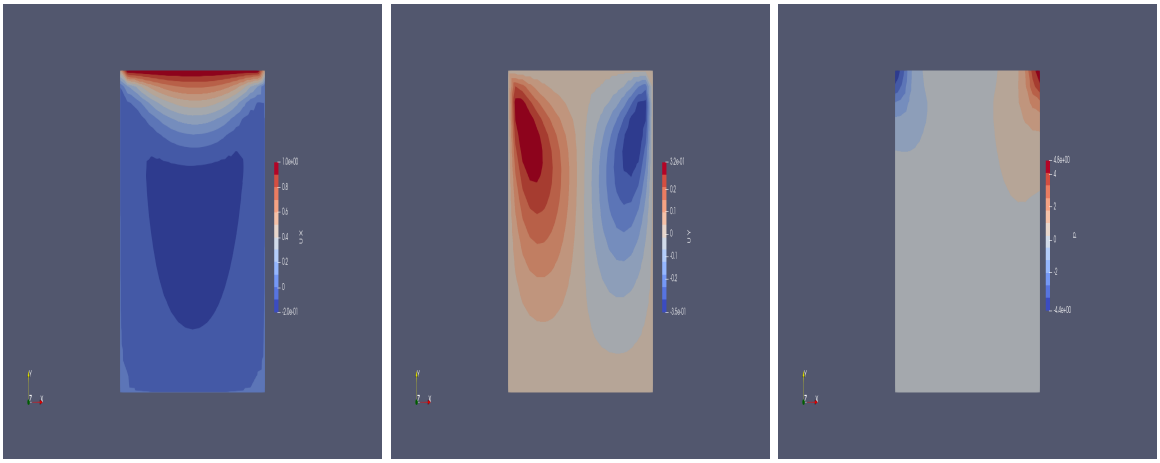

Figure 7: These contour plots shows the solutions by the OpenFoam with (i) Horizontal Velocity, (ii) Vertical Velocity and (iii) Pressure in the internal mesh with uniform grading and a resolution of 20**x**20

The above solutions are generated using the openFoam's `icoFoam` solver.

5 GenAI Disclosure

The authors declare that no generative AI tools were used in any part of the analysis, implementation, or writing process of this manuscript.

No tasks were delegated to generative AI systems under the GAIDeT taxonomy (2025) or any other framework.

The entire work was conducted independently by the authors without the assistance of tools such as ChatGPT, Claude, DeepSeek, Gemini or any other generative AI systems.

Declaration submitted by: Aditya Saroha

Additional note: I have not used AI in any part of this report and its associated implementation.